

CS 170 Programming Proficiency Quiz 5



For **PQ05 - Strings Parsing, Formatting and Functions**, you'll write two methods that take arguments and return values. This quiz covers the last parts of Chapters 3 and 4. For this quiz you'll need to know:

- The correct syntax for defining a method (writing the skeleton) which is about 25% of the grade.
- How to declare parameters correctly (type and order)
- How to call the methods in the `String` class (`substring`, `indexOf`, `charAt`, `length`)
- How to correctly return values from a method
- How to using parsing to convert `String` objects to integers or real numbers.
- How to produce formatted output, using the `String.format()` method (like `printf()`).

There is no `printf()` or printing at all on this test. There are also no loops, `if` statements or `Scanner` objects. You will not read any input. Each of the two methods will be tested for syntax correctness (name, `public`, return type and parameters) and for correctness. Work step-by-step so that you get partial credit.

You can find more problems to practice with on Codingbat and on Practicit. You'll find the links on Canvas. ***The following are a sample of the kind of problems you'll find, not necessarily the problems themselves.***

1 THE OUTSIDEIN3 PROBLEM

Write the method named `outsideIn3`. The method has one input: a `String str` that is at least 3 characters long and of odd length. The output of the method is a new `String` where

the middle character has been removed and placed both at the beginning and end. Here is some sample input and output:

- `outsideIn3("Cartons")`→ `"tCaronst"` // *t removed and placed at both ends*
- `outsideIn3("Hip")`→ `"iHpi"` // *i removed placed at both ends.*
- `outsideIn3("Barbara Ann")`→ `"rBarbaa Annr"` // *2nd r removed*

2 THE INSIDEOUT3 PROBLEM

Write the method named `insideOut3`. The method has one input: a `String str` that is at least 3 characters long and of odd length. The output of the method is a new `String` where the middle three characters have been removed. Here is some sample input and output:

- `insideOut3("Cartons")`→ `"Cans"` // *rto removed*
- `insideOut3("Hip")`→ `""` // *all three characters removed.*
- `insideOut3("Barbara Ann")`→ `"Barb Ann"` // *ara removed*

3 THE OUTSIDEIN2 PROBLEM

Write the method named `outsideIn2`. The method has one input: a `String str` that is at least 2 characters long and of even length. The output of the method is a new `String` where the middle two characters have been removed and placed at the beginning of the result. Here is some sample input and output:

- `outsideIn2("Carton")`→ `"rtCaon"` // *rt moves to front*
- `outsideIn2("Hi")`→ `"Hi"` // *Hi moved to from so output unchanged*
- `outsideIn2("Barbara Anne")`→ `"raBarba Anne"` // *ra moved to front*

4 THE LEFTN PROBLEM

Write the method `leftN`. The method has two inputs: a `String str` and an `int n`. The output of the method is a new `String` that contains a "rotated left n" version of `str`, where the first `n` chars are moved to the end of the `String`. The original `String` length will be at least `n`. Here is some sample input and output:

- `leftN("Hello", 2)` → `"lloHe"`
- `leftN("Chocolate", 3)` → `"colateCho"`
- `leftN("Chocolate", 1)` → `"hocolateC"`

5 THE MIDDLETHREE PROBLEM

Write the method `middleThree`. The method has one input: an odd-length `String str` of at least length 3. The output of the method is a new `String` that contains the middle three

characters, so calling the method with the String "Candy" returns "and". Here is some sample input and output:

- `middleThree("Candy")` → "and"
- `middleThree("and")` → "and"
- `middleThree("solving")` → "lvi"

6 THE NTWICE PROBLEM

Write the method `nTwice`. The method has two inputs: a String `str` and an `int n`. The output of the method is a new String made of the first and last `n` chars from `str`, where the last `n` chars are moved to the front of the String. The original String length will be at least length `n`. Here is some sample input and output:

- `rightN("Hello", 2)` → "loHel"
- `rightN("Chocolate", 3)` → "ateChocol "
- `rightN("Chocolate", 1)` → "eChocolat"

7 THE RIGHTN PROBLEM

Write the method `rightN`. The method has two inputs: a String `str` and an `int n`. The output of the method is a new String that contains a "rotated right `n`" version of `str`. You can assume that the length of `str` will be at least `n`. Here is some sample input and output:

- `nTwice("Hello", 2)` → "Helo"
- `nTwice("Chocolate", 3)` → "Choate"
- `nTwice("Chocolate", 1)` → "Ce"

8 THE INSIDEOUT2 PROBLEM

Write the method named `insideOut2`. The method has one input: a String `str` that is at least 2 characters long and of even length. The output of the method is a new String where the middle two characters have been removed and placed at the end of the result. Here is some sample input and output:

- `insideOut2 ("Carton")`→ "Caonrt" // rt moved to end
- `insideOut2 ("Hi")`→ "Hi" // *Hi moved to end, so no change.*
- `insideOut2 ("Barbara Anne")`→ "Barba Annera" //ra moves end.

9 THE GETDIMES PROBLEM

Write the method named `getDimes()`. The method has one input, a String named `amountStr` which consists of a dollar-formatted amount, such as "\$ 1,256.86". The

returned value is an `int` representing the number of "dimes" (10 cents) in the amount. You will need to scrub, format and parse the input, then use arithmetic to determine how many dimes the amount contains.

10 THE GETQUARTERS PROBLEM

Write the method named `getQuarters()`. The method has one input, a `String` named `amountStr` which consists of a dollar-formatted amount, such as "\$ 1,256.86". The returned value is an `int` representing the number of "quarters" (25 cents) in the amount. You will need to scrub, format and parse the input, then use arithmetic to determine how many quarters the amount contains.

11 THE GETDOLLARS PROBLEM

Write the method named `getDollars()`. The method has one input, a `String` named `amountStr` which consists of a dollar-formatted amount, such as "\$ 1,256.86". The returned value is an `int` representing the number of "dollars" in the amount. You will need to scrub, format and parse the input, then use arithmetic to determine how many quarters the amount contains.

12 THE GETPENNIES PROBLEM

Write the method named `getPennies()`. The method has one input, a `String` named `amountStr` which consists of a dollar-formatted amount, such as "\$ 1,256.86". The returned value is an `int` representing the number of "pennies" (cents) in the amount. You will need to scrub, format and parse the input, then use arithmetic to determine how many quarters the amount contains.

13 THE GETNICKELS PROBLEM

Write the method named `getNickels()`. The method has one input, a `String` named `amountStr` which consists of a dollar-formatted amount, such as "\$ 1,256.86". The returned value is an `int` representing the number of "nickels" (5 cents) in the amount. You will need to scrub, format and parse the input, then use arithmetic to determine how many quarters the amount contains.

14 THE ROUNDEDOLLARS PROBLEM

Write the method named `roundDollars()`. The method has one input, a `String` named `amountStr` which consists of a dollar-formatted amount, such as "\$ 1,256.86". The returned value is an `int` representing the number of rounded "dollars" in the amount (1257) in

the sample shown here. You will need to scrub, format and parse the input, then use arithmetic to determine how many quarters the amount contains.

15 THE LINE NUMBER PROBLEM

Write the method named `lineNumber()`. The method has two parameters, an `int` named `number` and a `String` named `text`. Return a formatted string where `number` is right-justified in a field 5 characters wide, followed by a colon and space, followed by the text. For example:

- `lineNumber(5, "Orange Coast College")` → " 5: Orange Coast College"

16 THE LINE ITEM PROBLEM

Write the method named `lineItem()` that formats an item on a receipt. The method has two parameters, a `String` `description` and a `double` `price`. Return a formatted string where `description` is left-aligned in a column 15 wide and `price` is right-aligned in a column 10 wide with two decimals. Thousands should be separated by commas but there should be no dollar sign. For example:

- `lineItem("Toaster", 24)` → "Toaster 24.00"

17 THE CHANGE PROBLEM

Write the method named `change()` that returns a list of coins, formatted in `String` form, returned from a purchase. The method has two parameters, the `amountTendered` and the `purchasePrice`, both `doubles`. Your method calculates the change due, and then formats the number of dollars, quarters(.25), dimes(.10), nickels(.05) and pennies(.01) due. Each number will be an `int` and it will be formatted in a column 5 wide. (In other words, your output will have exactly 5 columns and your returned `String` will be 50 characters long.